

Chapter 4: Threads & Concurrency

Comprehensive Study Notes with Examples, Code & Practice Questions

Operating System Concepts (Silberschatz, 10th Edition)

1. Overview: What is a Thread?

A **thread** is the basic unit of CPU utilization. It comprises a thread ID, a program counter (PC), a register set, and a stack. It shares with other threads belonging to the same process its code section, data section, and OS resources (open files, signals).

1.1 Components of a Thread vs. Process

Term	Definition
Thread ID	Unique identifier for each thread within a process
Program Counter	Points to the next instruction to execute for that thread
Register Set	Working variables; each thread has its own registers
Stack	Stores local variables, return addresses — private to each thread
Shared Code	All threads in a process share the same code segment
Shared Data	Global variables and heap are shared among all threads
Shared OS Resources	Open file descriptors, signals etc. are shared

Key Analogy: Think of a process as a factory and threads as workers inside that factory. Workers share the same tools and workspace (code, data, files) but each has their own task list and clipboard (stack, PC, registers).

1.2 Benefits of Multithreading

Benefit	Explanation	Example
Responsiveness	Execution continues even if one part blocks	Web browser downloads in background while UI stays active
Resource Sharing	Threads share memory of the process easily	All threads access same global data without IPC
Economy	Creating a thread is ~30x cheaper than process creation	Web server spawns thread per request instead of process
Scalability	Threads run on multiple CPU cores simultaneously	Scientific computation distributes work across all cores

2. Multicore Programming

2.1 Concurrency vs. Parallelism

Concurrency: Multiple tasks make progress. Can occur on a single-core system via time-sharing (scheduler switches rapidly between tasks).

Parallelism: Multiple tasks execute simultaneously. Requires a multicore/multiprocessor system.

NOTE: KEY DISTINCTION: You can have concurrency WITHOUT parallelism (single-core), but parallelism ALWAYS implies concurrency.

Feature	Concurrency	Parallelism
Definition	Multiple tasks making progress	Multiple tasks executing at the same time
Hardware needed	Single or multi-core	Must have multiple cores/CPUs
How achieved	Rapid context switching by scheduler	True simultaneous execution on separate cores
Example	OS juggling multiple apps on 1-core CPU	4-core CPU running 4 matrix operations at once

2.2 Types of Parallelism

- Data Parallelism: Distribute subsets of the same data across multiple cores, performing the same operation on each subset. Example: Summing a large array by splitting it into 4 parts for 4 cores.
- Task Parallelism: Distribute different tasks (threads) across cores. Each thread performs a unique operation. Example: One thread compresses data, another encrypts it, simultaneously.

2.3 Challenges in Multicore Programming

- Dividing Activities: Identifying which tasks can run in parallel
- Balance: Ensuring each parallel unit does equal work
- Data Splitting: Dividing data for access by parallel tasks
- Data Dependency: Task A must finish before Task B can start — limits parallelism
- Testing and Debugging: Errors may not be reproducible; race conditions appear randomly

3. Amdahl's Law

Amdahl's Law predicts the maximum speedup achievable by parallelizing a program, given that some portion must remain serial.

$$\text{speedup} \leq 1 / (S + (1-S)/N)$$

Where: S = serial fraction of program, N = number of processing cores

3.1 Key Observations

- As N approaches infinity, speedup approaches 1/S (upper bound)
- The serial portion S has a DISPROPORTIONATE effect on performance
- Even with infinite cores, a 20% serial portion limits speedup to 5x maximum

3.2 Numerical Examples

Scenario	Serial (S)	Parallel (1-S)	Max Speedup (N=inf)	Speedup at N=2
Program A	0.20 (20%)	0.80 (80%)	$1/0.20 = 5x$	$1/(0.20 + 0.80/2) = 1.67x$
Program B	0.10 (10%)	0.90 (90%)	$1/0.10 = 10x$	$1/(0.10 + 0.90/2) = 1.82x$
Program C	0.05 (5%)	0.95 (95%)	$1/0.05 = 20x$	$1/(0.05 + 0.95/2) = 1.90x$
Program D	0.50 (50%)	0.50 (50%)	$1/0.50 = 2x$	$1/(0.50 + 0.50/2) = 1.33x$

3.3 Worked Example

Problem: A program has 25% serial code and 75% parallelizable code. Calculate speedup for N=2, N=4, and N=infinity cores.

Given: $S = 0.25$, $(1-S) = 0.75$

N = 2 cores:

$\text{speedup} = 1 / (0.25 + 0.75/2) = 1 / (0.25 + 0.375) = 1 / 0.625 = 1.6x$

N = 4 cores:

$\text{speedup} = 1 / (0.25 + 0.75/4) = 1 / (0.25 + 0.1875) = 1 / 0.4375 = 2.29x$

N = infinity:

$\text{speedup} = 1 / S = 1 / 0.25 = 4.0x$ (theoretical maximum)

4. Multithreading Models

The relationship between user threads (managed by thread libraries) and kernel threads (managed by the OS) defines the threading model.

4.1 Many-to-One Model

- Many user threads mapped to ONE kernel thread
- Thread management done in user space — efficient
- PROBLEM: If one thread makes a blocking system call, the ENTIRE process blocks
- Cannot exploit multicore parallelism (only one kernel thread)
- Examples: Solaris Green Threads, GNU Portable Threads

NOTE: Rarely used today because it cannot take advantage of multicore systems.

4.2 One-to-One Model

- Each user thread maps to a SEPARATE kernel thread
- More concurrency — one thread blocking does not affect others
- Better performance on multicore systems (true parallelism)
- **DRAWBACK:** Overhead of creating kernel thread for each user thread; number may be restricted
- Examples: Linux (pthreads), Windows

NOTE: Most modern operating systems (Linux, Windows) use One-to-One model.

4.3 Many-to-Many Model

- Many user threads multiplexed to EQUAL OR FEWER kernel threads
- Balances concurrency and resource usage
- No true blocking problem: OS can schedule another kernel thread
- Complex to implement; less relevant with abundant cores today
- Example: Windows with ThreadFiber package

4.4 Two-Level Model (Variant of M:M)

- Similar to Many-to-Many but also allows some user threads to be BOUND to a specific kernel thread
- Provides flexibility: critical threads get dedicated kernel threads

Model	Mapping	Blocking?	Parallelism	Used In
Many-to-One	Many UTs → 1 KT	Yes — all block	No	Old systems
One-to-One	1 UT → 1 KT	No — others continue	Yes	Linux, Windows
Many-to-Many	Many UTs → ≤N KTs	Partial	Yes	Rare (ThreadFiber)
Two-Level	M:M + some bound	Partial	Yes	Specialized

5. Thread Libraries & Pthreads

5.1 Overview

Thread libraries provide programmers with an API to create and manage threads. Implemented either entirely in user space or as a kernel-level library.

- POSIX Pthreads (Linux/macOS) — kernel-level

- Windows Threads API — kernel-level
- Java Threads — managed by JVM, maps to OS threads

5.2 POSIX Pthreads — Complete API Reference

Function	Purpose	Key Parameters
pthread_create()	Create a new thread	thread_id, attr, start_fn, arg
pthread_exit()	Terminate calling thread	retval (return value)
pthread_join()	Wait for a thread to terminate	thread_id, status (return value)
pthread_cancel()	Send cancellation request to thread	thread_id
pthread_detach()	Detach thread (auto-cleanup on exit)	thread_id
pthread_attr_init()	Initialize thread attributes	attr pointer
pthread_mutex_lock()	Acquire mutex lock	mutex pointer
pthread_mutex_unlock()	Release mutex lock	mutex pointer
pthread_kill()	Send signal to specific thread	thread_id, signal_number

5.3 Example 1: Basic Thread Creation

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

// Thread function — executed by the new thread
void *helloWorld(void *vargp) {
    sleep(1); // Simulate some work
    printf("Hello World from thread!\n");
    return NULL;
}

int main() {
    pthread_t thread_id;

    printf("Before Thread\n");

    // Create thread — thread runs helloWorld()
    pthread_create(&thread_id, NULL, helloWorld, NULL);

    // Wait for thread to finish
    pthread_join(thread_id, NULL);

    printf("After Thread\n");
    exit(0);
}

/* Output:
Before Thread
Hello World from thread!
After Thread
*/
```

5.4 Example 2: Computing Sum Using a Thread

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

int sum; // shared global - thread writes here

void *runner(void *param) {
    int i, upper = atoi(param);
    sum = 0;
    for (i = 1; i <= upper; i++)
        sum += i;
    pthread_exit(0);
}

int main(int argc, char *argv[]) {
    pthread_t tid;
    pthread_attr_t attr;

    pthread_attr_init(&attr); // default attributes
    pthread_create(&tid, &attr, runner, argv[1]);
    pthread_join(tid, NULL); // wait for thread

    printf("sum = %d\n", sum);
    return 0;
}

/* Run: ./program 10 => sum = 55 */
```

5.5 Example 3: Multiple Threads with Fork-Join (Parallel Array Sum)

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ARRAY_SIZE 1000000

int global_array[ARRAY_SIZE];

void *sum_array(void *arg) {
    int thread_id = *((int *)arg);
    int start = thread_id * (ARRAY_SIZE / NUM_THREADS);
    int end = start + (ARRAY_SIZE / NUM_THREADS);
    long sum = 0;
    for (int i = start; i < end; i++)
        sum += global_array[i];
    return (void *) (long)sum; // return partial sum
}

int main() {
    pthread_t threads[NUM_THREADS];
    int thread_args[NUM_THREADS];
    void *thread_results[NUM_THREADS];
    long total_sum = 0;

    // Initialize array
    for (int i = 0; i < ARRAY_SIZE; i++) global_array[i] = rand() % 100;
```

```

// FORK: create threads
for (int i = 0; i < NUM_THREADS; i++) {
    thread_args[i] = i;
    pthread_create(&threads[i], NULL, sum_array, &thread_args[i]);
}

// JOIN: collect results
for (int i = 0; i < NUM_THREADS; i++) {
    pthread_join(threads[i], &thread_results[i]);
    total_sum += (long)thread_results[i];
}

printf("Total sum: %ld\n", total_sum);
return 0;
}

```

5.6 Thread Termination Methods

1. Return normally from start routine
2. Call `pthread_exit(retval)` — thread-specific exit
3. Cancelled by another thread via `pthread_cancel(tid)`
4. Entire process calls `exec()` or `exit()`
5. `main()` finishes without calling `pthread_exit()` — all other threads abruptly end

NOTE: `pthread_cancel()` only REQUESTS cancellation. The thread is not guaranteed to stop immediately. Use `pthread_join()` afterward to ensure cleanup.

6. Implicit Threading

As thread count increases, managing threads manually becomes error-prone. Implicit threading delegates thread creation and management to compilers and runtime libraries.

6.1 Fork-Join Model

Main thread FORKS into multiple worker threads, then JOINS (waits) for all to complete before continuing. The Pthreads parallel array sum example above demonstrates this pattern.

6.2 Thread Pools

- Create a fixed pool of threads at startup — no creation overhead per request
- Tasks placed in a queue; idle threads pick up tasks
- Controls maximum thread count, preventing system overload
- Used in web servers: Apache, Nginx use thread pools for handling requests

7. Threading Issues

7.1 `fork()` and `exec()` Semantics

- When a thread calls `fork()`: Does it duplicate only the calling thread, or all threads?
- Linux: Two versions of `fork()` exist — one duplicates all threads, another only the caller
- `exec()` always replaces the ENTIRE process (all threads) with the new program
- Rule of thumb: If `exec()` immediately follows `fork()`, duplicate only the calling thread

7.2 Signal Handling

Signals notify a UNIX process that an event has occurred (e.g., division by zero, Ctrl+C).

Signal Type	Description	Example
Synchronous	Generated by the thread's own action; delivered to same thread	Illegal memory access, division by zero (SIGSEGV, SIGFPE)
Asynchronous	Generated by external event; sent to any thread	Ctrl+C (SIGINT), timer expiry (SIGALRM)

Options for delivering signals in multithreaded programs:

6. Deliver to the specific thread to which the signal applies
7. Deliver to every thread in the process
8. Deliver to certain threads (those that haven't blocked it)
9. Assign a dedicated thread to receive all signals

7.3 Thread Cancellation

Terminating a thread before it has finished. The thread being cancelled is the target thread.

Type	Behaviour	Risk	When to Use
Asynchronous Cancellation	Thread terminated IMMEDIATELY	May leave resources unreleased, data inconsistent	Rarely — only for stateless threads
Deferred Cancellation (default)	Thread checks cancellation at defined cancellation points	Safer — cleanup handlers can run	Most cases — pthread default

```
// Deferred cancellation example
pthread_t tid;
pthread_create(&tid, 0, worker, NULL);

// ... later ...
pthread_cancel(tid); // request cancellation
pthread_join(tid, NULL); // wait for cleanup
```

7.4 Thread-Local Storage (TLS)

- Allows each thread to have its OWN COPY of data — not shared with other threads
- Unlike local variables: TLS persists across multiple function calls within the thread
- Unlike global/static: TLS is unique per thread
- Useful with thread pools where you can't control thread creation


```
// Declaration of TLS variable in C (GCC extension)
__thread int threadID;    // Each thread has its own 'threadID'

// Or using pthread keys:
pthread_key_t key;
pthread_key_create(&key, destructor_fn);
pthread_setspecific(key, value);    // set TLS value
pthread_getspecific(key);           // get TLS value
```

8. Operating System Thread Implementations

8.1 Linux Threads

- Linux refers to threads as TASKS (not threads internally)
- Uses the clone() system call to create threads — not fork()
- clone() allows child task to share parent's address space

```
// Linux clone() flags control what is shared:
// CLONE_FS      - share file-system information
// CLONE_VM      - share same memory space (makes it a thread)
// CLONE_SIGHAND - share signal handlers
// CLONE_FILES   - share open file descriptors

// When all flags set → behaves like pthread_create()
// When no flags set → behaves like fork() (separate process)
```

8.2 User-Level vs. Kernel-Level Thread Scheduling

On modern OSes, the kernel schedules KERNEL-LEVEL threads. User-level threads must be mapped to kernel threads to run.

Scope	Name	Contention Among	Scheduled By
Process Contention Scope (PCS)	PTHREAD_SCOPE_PROCESS	Threads within same process for LWPs	User-level library
System Contention Scope (SCS)	PTHREAD_SCOPE_SYSTEM	All threads across entire system	OS kernel scheduler

9. Race Conditions — Introduction

When multiple threads access shared data and the outcome depends on execution order, a RACE CONDITION exists. This is a critical issue that motivates synchronization (Chapter 6).

```
// RACE CONDITION EXAMPLE
static volatile int counter = 0;

void *mythread(void *arg) {
    for (int i = 0; i < 1e7; i++)
        counter = counter + 1; // NOT atomic! 3 steps:
    return NULL;               // 1. load counter
}                             // 2. increment
                             // 3. store counter

// If Thread A loads counter=5, Thread B loads counter=5,
```

```
// both increment to 6, both store 6 – net: +1 instead of +2!  
// Expected: 20,000,000 Actual: UNPREDICTABLE
```

NOTE: Solution: Use synchronization mechanisms like Mutexes, Semaphores, Monitors (covered in Chapter 6).

10. Exam Practice Questions

PART A: Theory & Reasoning Questions

Short Answer / Reasoning Questions

Q: What are the four basic components of a thread? How does a thread differ from a process?

A: A thread has: (1) Thread ID, (2) Program Counter, (3) Register Set, (4) Stack. Unlike a process, threads within the same process SHARE code section, data section, heap, and OS resources. A process has its own separate memory space. Threads are 'lightweight processes' — cheaper to create/switch.

Q: Explain the four benefits of multithreading with real-world examples.

A: (1) Responsiveness: A word processor continues responding to keyboard input while spell-checking in background. (2) Resource Sharing: Threads in a web server share a connection pool without complex IPC. (3) Economy: Apache creates a thread per request (not a process) — ~30x cheaper creation overhead. (4) Scalability: A scientific simulation divides work across all CPU cores simultaneously.

Q: Distinguish between data parallelism and task parallelism with examples.

A: Data Parallelism: The SAME operation applied to different subsets of data simultaneously. Example: Adding two 1-million-element arrays — split into 4 chunks, each core adds its 250K elements. Task Parallelism: DIFFERENT operations run concurrently. Example: Thread 1 compresses an image while Thread 2 uploads the previous one — different operations on different/same data.

Q: Why is the Many-to-One threading model rarely used in modern systems?

A: Because: (1) If ONE user thread makes a blocking system call, the ENTIRE process blocks — all other threads are stuck. (2) Only ONE kernel thread exists for the process, so threads CANNOT run in parallel on multicore systems. Modern systems need both non-blocking behavior and multicore utilization, which Many-to-One cannot provide.

Q: Explain the difference between asynchronous and deferred thread cancellation. Which is safer and why?

A: Asynchronous Cancellation: The target thread is terminated IMMEDIATELY when cancellation is requested. Unsafe because the thread may be in the middle of updating shared data or holding a lock, causing data corruption or deadlock. Deferred Cancellation (default in pthreads): The thread checks for cancellation only at specific 'cancellation points' (e.g., `sleep()`, `pthread_cond_wait()`). It can run cleanup handlers before terminating, making it SAFER. Deferred is the preferred approach.

Q: What is Thread-Local Storage (TLS)? How does it differ from local variables and static variables?

A: TLS gives each thread its own private copy of a variable. Comparison: (1) Local Variables: Exist only during one function call, stack-based — TLS persists across calls. (2) Static Variables: ONE copy shared by all threads — TLS has one copy PER THREAD. Use case: A thread pool where each worker needs its own transaction ID — use TLS so each thread has its own copy without sharing.

Q: How does Linux implement threads? What is the key difference from `fork()`?

A: Linux uses the clone() system call. Unlike fork() (which creates an independent process with separate memory), clone() allows the child task to SHARE the parent's address space and other resources. Flags control what is shared: CLONE_VM (memory), CLONE_FS (file system), CLONE_FILES (open files), CLONE_SIGHAND (signals). Setting all these flags effectively creates a thread; setting none creates an independent process (like fork).

Q: A program has a thread that calls fork(). Should fork() duplicate only the calling thread or all threads? When should each behavior be used?

A: This is OS-dependent — Linux provides both versions. Rule: If exec() is called immediately after fork(), duplicate only the calling thread (other threads would be discarded by exec() anyway — no point duplicating them). If the child process will NOT call exec() and needs to continue multi-threaded operation, all threads should be duplicated. In practice, forking in a multithreaded program is discouraged because it leads to complex, hard-to-manage states.

Q: Explain the concept of a race condition using the counter++ example from the slides.

A: counter++ is NOT atomic — it compiles to 3 steps: (1) Load counter into register, (2) Increment register, (3) Store register back to counter. If Thread A loads counter=5, then the OS context-switches to Thread B which also loads counter=5 and stores 6, then Thread A stores 6 — the increment from Thread A is LOST. With 2 threads each doing 10M increments, the expected result is 20M, but actual result is unpredictably less. This is a race condition — result depends on thread scheduling timing.

Numerical / Calculation Questions (Amdahl's Law)

Q1. A program has 60% parallelizable code. Calculate the speedup for N=2, N=4, N=8 cores. What is the theoretical maximum speedup?

Solution: S = 0.40 (40% serial)

N=2: speedup = $1 / (0.40 + 0.60/2) = 1 / (0.40 + 0.30) = 1/0.70 = 1.43x$

N=4: speedup = $1 / (0.40 + 0.60/4) = 1 / (0.40 + 0.15) = 1/0.55 = 1.82x$

N=8: speedup = $1 / (0.40 + 0.60/8) = 1 / (0.40 + 0.075) = 1/0.475 = 2.11x$

N=inf: speedup = $1/S = 1/0.40 = 2.5x$ (maximum)

Note: The 40% serial portion severely limits gains. Even with infinite cores, you only get 2.5x speedup.

Q2. You want a speedup of at least 4x using 8 cores. What fraction of the code must be parallelizable?

Solution: We need speedup ≥ 4 with N=8

$4 \leq 1 / (S + (1-S)/8)$

$S + (1-S)/8 \leq 0.25$

$8S + (1-S) \leq 2.0$ [multiply both sides by 8]

$8S + 1 - S \leq 2.0$

$7S \leq 1.0$

$S \leq 1/7 \approx 0.143$ (14.3%)

Therefore at least 85.7% of the code must be parallelizable.

Q3. Program A is 90% parallel (N=4). Program B is 75% parallel (N=8). Which gives better speedup? Justify.

Program A (S=0.10, N=4):

speedup = $1 / (0.10 + 0.90/4) = 1 / (0.10 + 0.225) = 1/0.325 = 3.08x$

Program B (S=0.25, N=8):

$\text{speedup} = 1 / (0.25 + 0.75/8) = 1 / (0.25 + 0.094) = 1/0.344 = 2.91x$
Program A gives better speedup (3.08x > 2.91x)!
Lesson: Even doubling cores doesn't overcome a larger serial fraction.

Q4. A video rendering program is 95% parallelizable. Calculate speedup for N=2,4,8,16 cores and explain the diminishing returns.

S = 0.05 (5% serial)

N=2: $1 / (0.05 + 0.95/2) = 1/0.525 = 1.90x$

N=4: $1 / (0.05 + 0.95/4) = 1/0.2875 = 3.48x$

N=8: $1 / (0.05 + 0.95/8) = 1/0.1688 = 5.92x$

N=16: $1 / (0.05 + 0.95/16) = 1/0.1094 = 9.14x$

N=inf: $1/0.05 = 20x$ (maximum)

Doubling from N=2 to N=4 gives +1.58x gain. Doubling N=8 to N=16 gives only +3.22x. Returns diminish because the 5% serial portion becomes the bottleneck.

Q5. (Tricky) A program currently runs in 100 seconds on 1 core. 80 seconds is parallelizable. How long does it run on 4 cores? On 16 cores?

Serial time = 20s, Parallel time = 80s, Total = 100s

$S = 20/100 = 0.20$ $(1-S) = 0.80$

On N=4 cores: $\text{speedup} = 1 / (0.20 + 0.80/4) = 1/0.40 = 2.5x$

Runtime = $100 / 2.5 = 40$ seconds

On N=16 cores: $\text{speedup} = 1 / (0.20 + 0.80/16) = 1/0.25 = 4.0x$

Runtime = $100 / 4.0 = 25$ seconds

The 20-second serial portion means the best possible time is 20s (adding infinite cores won't help beyond that).

Code Tracing Questions

Q6. What are all possible outputs of this Pthreads program? Explain why output order varies:

```
void *mythread(void *arg) {
    printf("%s\n", (char *) arg);
    return NULL;
}

int main() {
    pthread_t p1, p2;
    pthread_create(&p1, NULL, mythread, "A");
    pthread_create(&p2, NULL, mythread, "B");
    pthread_join(p1, NULL);
    pthread_join(p2, NULL);
    printf("main: end\n");
    return 0;
}
```

Answer: Three possible outputs:

Output 1: A, B, main:end (T1 runs first, then T2)

Output 2: B, A, main:end (T2 runs first, then T1)

Output 3: A, B already done before joins, main:end (T1 runs to completion before T2 starts)

Reason: Thread scheduling is non-deterministic. The OS decides which thread runs when. main:end always comes last because pthread_join() waits for both threads.

Q7. Identify the problem in this code and explain what could go wrong:

```
int counter = 0;

void *increment(void *arg) {
    for (int i = 0; i < 1000000; i++)
        counter++;          // Problem here!
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("counter = %d\n", counter);    // Expected: 2000000
    return 0;
}
```

Answer: RACE CONDITION on 'counter++'

counter++ is 3 instructions: LOAD, ADD, STORE. If T1 loads counter=500000, T2 also loads 500000, T2 stores 500001, T1 stores 500001 — one increment is lost.

Actual output is unpredictably less than 2,000,000 and varies each run.

Fix: Use pthread_mutex_lock/unlock around counter++ to make it atomic.

Multiple Choice Questions (MCQs)

Q8. Which component is NOT shared between threads of the same process?

- A) Code segment
- B) Data segment (global variables)
- C) Stack
- D) Open file descriptors

Answer: C) Stack — Each thread has its own private stack for local variables and function calls.

Q9. In the One-to-One threading model, what happens when one thread makes a blocking system call?

- A) All threads in the process block
- B) Only the calling thread blocks; others continue
- C) The entire system halts
- D) The thread is terminated

Answer: B) In One-to-One, each user thread has its own kernel thread, so only the calling kernel thread blocks. Other kernel threads (and their user threads) continue running.

Q10. According to Amdahl's Law, a program with 50% serial code can achieve a maximum speedup of:

- A) 4x

- B) 2x
- C) Infinite
- D) 8x

Answer: B) 2x — Maximum speedup = $1/S = 1/0.50 = 2x$ regardless of how many cores are added.

Q11. `pthread_join()` serves what purpose?

- A) Creates a new thread
- B) Terminates the calling thread
- C) Waits for a specified thread to terminate
- D) Sends a signal to a thread

Answer: C) `pthread_join()` blocks the calling thread until the specified thread terminates, then collects its exit status.

Q12. Linux internally creates threads using which system call?

- A) `fork()`
- B) `exec()`
- C) `clone()`
- D) `pthread_create()`

Answer: C) `clone()` — Linux uses `clone()` with appropriate flags (`CLONE_VM`, `CLONE_FS`, etc.) to create threads that share the parent's address space. `pthread_create()` is a library call that internally uses `clone()`.

Quick Reference Summary Card

Amdahl's Law Formula

$$\text{speedup} \leq 1 / (S + (1-S)/N)$$

Max speedup (N→∞) = 1/S | S = serial fraction | N = number of cores

Threading Models Comparison

Model	User→Kernel	True Parallelism?	Blocking Issue?	Examples
Many-to-One	Many → 1	No	Yes — all block	Solaris Green Threads
One-to-One	1 → 1	Yes	No	Linux, Windows
Many-to-Many	Many → ≤N	Yes	Partial	Windows ThreadFiber
Two-Level	M:M + bound	Yes	Partial	Specialized OS

Key Pthreads Functions

Function	What it does
pthread_create(&tid, attr, fn, arg)	Create thread running fn(arg)
pthread_join(tid, &status)	Wait for thread tid to finish
pthread_exit(retval)	Terminate current thread with return value
pthread_cancel(tid)	Request cancellation of thread tid
pthread_detach(tid)	Auto-cleanup thread resources on exit
pthread_mutex_lock(&m)	Acquire exclusive lock m
pthread_mutex_unlock(&m)	Release lock m
pthread_kill(tid, sig)	Send signal sig to thread tid

Key Terms to Know

Term	One-Line Definition
Thread	Lightweight unit of CPU execution — shares process resources
Concurrency	Multiple tasks making progress (not necessarily simultaneously)
Parallelism	Multiple tasks executing at the exact same time (multi-core)
Race Condition	Bug where result depends on unpredictable thread scheduling
TLS	Per-thread private data that persists across function calls
Amdahl's Law	Formula bounding max speedup based on serial code fraction
Fork-Join	Pattern: fork threads for parallel work, join to collect results

Thread Pool	Pre-created threads waiting for work — avoids creation overhead
-------------	---

— *End of Chapter 4 Study Notes* —